



Improving the State of Automotive Software Engineering

Alireza Haghighatkah, Markku Oivo, Ahmad Banijamali, and Pasi Kuvaja

THE AUTOMOTIVE INDUSTRY is fundamentally changing by becoming software intensive, rather than mechanically intensive. Now, innovation and competitiveness rely principally on software engineering competence. You could argue that today's premium vehicles run on code as much as on fuel.

Software and electronics in vehicles have led to innovative functionalities; reduced fuel consumption; and improved performance, comfort, and safety.¹ Moreover, software has negligible replication costs and enables reuse, mass differentiation, and customization.¹ Thus, software engineering gives automakers a competitive advantage.

To stay ahead of the game, industry players must continuously improve software engineering technologies, as well as their underlying processes and practices. The literature contains much knowledge that could help automakers improve industry practices. Via an extensive systematic mapping study, we aggregated and synthesized information from academic and practitioner-oriented literature, analyzed the material, and developed a set of practitioner-oriented recommendations.

Literature Review

To find studies on automotive software engineering (ASE) relevant to our

research, we searched five major scientific databases and the Society of Automotive Engineers' digital library of practitioner-oriented articles. From 4,348 retrieved records, we found 679 articles relevant to our scope, including 247 academic studies, 240 industry-academic collaborative works, and 192 practitioner-oriented articles. (The full list of analyzed studies and data extraction results is at www.dropbox.com/s/n7ix7h5yk9puavs/ASE_SMS_Repository_17022016.xlsx?dl=0). We then examined these articles carefully.²

The Automotive Domain's Characteristics

While the automotive industry shares features with other domains, it has its own characteristics, such as¹

- heterogeneous and distributed software,
- a distributed development environment,
- many automobile variants and configurations,
- cost pressure and the predominance of unit-based cost models, and
- the prevalence of legacy systems.

These characteristics yield several fundamental ASE challenges¹ and affect the automotive industry in

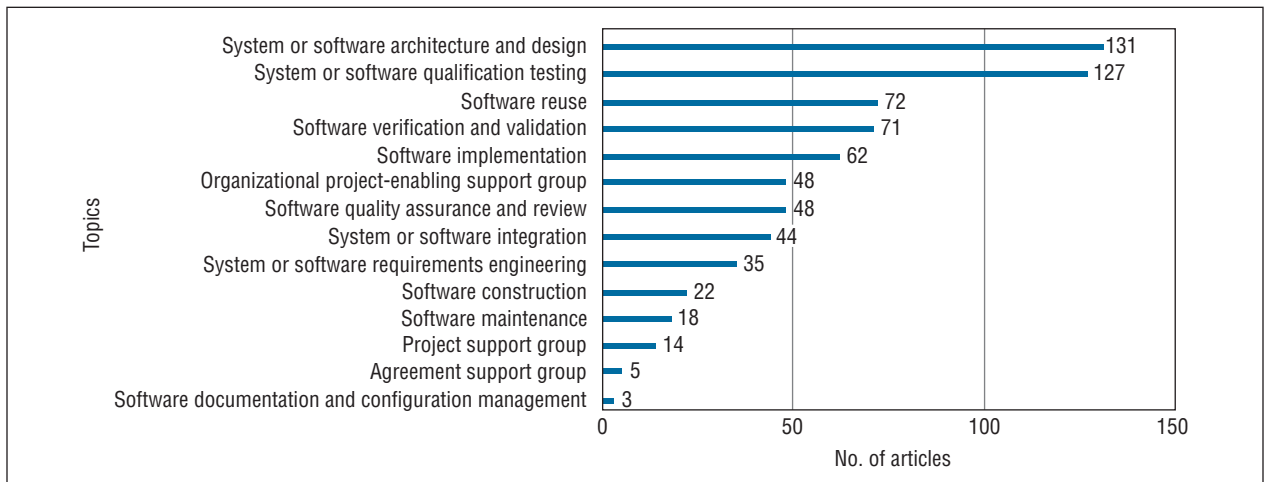


FIGURE 1. The automotive-software topics mentioned in the greatest number of author-reviewed articles.

general, in areas such as long-term strategies.

What Do We Know about ASE?

Figure 1 shows topics identified in the ASE literature and the number of articles that mentioned them, thereby revealing the main areas of concern. Three topics—system or software architecture and design, system or software qualification testing, and software reuse—were addressed most often.

AUTOSAR

To manage automotive-software development's complexity, the industry has developed and adopted several standards. For example, AUTOSAR (*Automotive Open System Architecture*) creates an open and standardized software architecture for vehicular electronic control units. Surveys show that AUTOSAR yields advantages such as standardization, efficient development, shorter development lead time with the ability to introduce improvements and new features faster, software reuse, and interoperability.^{3,4} It also has drawbacks such as

complexity; the initial investment in new processes, practices and tools; and a steep learning curve.

Automakers will have to migrate their existing legacy systems to an AUTOSAR-compliant architecture to avoid experiencing interoperability and compatibility problems, as well as hindering innovation.²

Testing

An empirical automotive-software-testing study reported 26 challenges and 15 solutions.⁵ The challenges primarily entail requirements, test management, and automation. The solutions involve requirements management, competence management, quality assurance and standards, test automation and tools, agile incorporation, and test management.

A particular problem deals with the ability to exchange test specifications among teams and systems. In this situation, the use of different test languages—which have various syntaxes, semantics, data formats, and interface descriptions—causes difficulties.⁶ To avoid this problem, practitioners must adopt formal, test-platform-independent protocols.

Existing evidence suggests that automatically and manually derived model-based test suites are much better at detecting requirement errors than handcrafted test suites.⁷ Findings show that automated test execution is heavily used but automated test generation isn't because current tools don't support it.⁸

Given the enormous range of products and variants, exhaustive testing of entire automotive software product lines (SPLs) is unrealistic. Several proposed solutions would derive small representative sets of test cases that could demonstrate the entire product line's correctness.⁹

In addition, continuous-controller testing is demanding because of its many inputs and configuration parameters. To deal with this, researchers have proposed several search-based testing approaches, which practitioners could use to automate complex-system verification.¹⁰

Requirements Engineering

Studies have found that requirements engineering (RE) is often inadequate in practice and that its processes aren't well defined.¹¹ Challenges

include the lack of a comprehensive requirements model, the unsystematic capture of requirements, the need to model a large number of requirements, the modeling of non-functional characteristics, requirements volatility, and inadequate tool support.¹

Natural-language requirements specification, although still common, is prone to problems for several reasons. Although structured natural-language specification affords some advantages, it doesn't support automated analysis, verification, and requirements transformation. The automotive industry should thus fully adopt model-based RE.¹¹

Studies have identified model-based development as a possible solution to many ASE challenges. However, automakers haven't fully exploited its potential because of the lack of both a unified development process and an integrated tool chain.¹ To deal with this issue, practitioners have taken a pragmatic, demand-driven, and ad hoc approach to adapting their engineering methods and processes to the available tool environments. As a result, the practice suffers from redundancy, inconsistency, and the lack of automation. To achieve a seamless model-based development environment, practitioners must adopt a comprehensive development methodology based on a well-established modeling theory, a common product model, an appropriate process model, and an integrated tool chain.¹

Automakers must also develop and carefully manage traceability across requirements artifacts and the rest of the development process. Previous studies have emphasized the importance of aligning RE and software testing and have

proposed mechanisms for their improved traceability, adjustment, and coordination.

Agile Practices

The rapid development and continuous delivery of software have become competitive advantages in the automotive industry. However, automakers have only just begun deploying agile development. Studies show that the key challenges to agile adoption in the industry involve transforming the organizational structure and culture, and achieving a shorter release cycle without compromising quality.¹² Other challenges include software reuse with agile practices, applying appropriate quality-assurance measures, and collaborating with suppliers and specialists in other disciplines, such as mechanics. To deal with these issues, researchers propose incremental, stepwise agile adoption.¹²

The demand for improved and innovative functions is increasing. At the same time, the technologies behind online diagnosis and over-the-air updates have matured. These developments create new opportunities for continuous delivery, effective recall management, and rapid innovation based on real-time data analytics.² This vision requires the development of infrastructure to support continuous integration (CI) and the continuous delivery of automotive systems.

The main concern in automotive system integration is synthesizing and deploying functions without compromising quality-related attributes such as performance and safety. Studies show that syntactical and technological integration-related challenges are typically addressed during the design phase by using a standardized architecture, while

other demanding integration issues are addressed late in the integration process.¹³ This leads to last-minute chaos and the late identification of integration issues. To avoid these problems, integration should be automated and performed frequently. To integrate as often and early as possible, practitioners should invest in virtualized integration platforms. This makes CI possible during the development process and minimizes dependency on other parts of the system. Because manual integration is an expensive and error-prone process, practitioners should automate and integrate the synthesis and deployment of software functions with CI systems.

CI systems enable more frequent and continuous integration of changes, which increases the need for optimized regression testing (RT). The "retest all" approach isn't feasible for automotive RT because of the cost and large number of test cases. Thus, practitioners must effectively minimize, select, and prioritize test cases. The software engineering literature has proposed many techniques to improve RT processes, but applying them in the automotive industry remains challenging because source code often isn't available.¹⁴ Thus, alternative approaches that exploit historical data and execution traces seem more appropriate.¹⁴

Software Quality

The articles we reviewed emphasized software reliability's importance and its implications for effective resource allocation, product quality improvement, and release-readiness evaluation. One set of researchers investigated the applicability of commonly used software reliability growth models

(SRGM) in embedded-software projects.¹⁵ Of the eight investigated SRGMs, they found that the Logistic and Gompertz models were the most accurate. They also found that the expected shape of defect inflows helps select the most appropriate SRGM model.

Another study investigated software defect-prediction models, their characteristics, and their applicability throughout the automotive software development lifecycle.¹⁶ This study's findings can help practitioners choose a defect-prediction model based on its characteristics and the availability of input data.

Modeling systems such as the Simulink graphical programming environment and the Stateflow control-logic tool are now important automotive-development assets. Empirical studies show that clone management of models fosters reuse and improves automotive-system quality and maintenance. Researchers have proposed solutions to help practitioners automatically identify clones in behavioral models.¹⁷ Several case studies have shown these approaches' effectiveness and scalability.

Variability and Reuse

The automotive industry must fulfill a range of legal requirements while addressing different markets via mass differentiation and customization. Variability can thus become complicated, leading to significant costs and risks. Currently, the industry is dealing with several software-reuse challenges.¹ Issues include the imprecise estimation of economic returns, the need to balance immediate customer needs with long-term benefits, variability-management complexity, quality concerns, organizational matters, the lack of a comprehensive

reuse strategy, and intensive software tailoring and optimization.

To establish a successful SPL, practitioners must employ visually informed variability management, standardized architecture, seamless model-driven development, and agile practices.¹⁸ Articles we reviewed support automated approaches for variability identification and management, the automatic diagnosis and debugging of product configurations, SPL test-selection mechanisms, and the automatic generation of modular SPL safety cases.

ASE research appears quite relevant for the automotive industry, as evidenced by the large number of empirical studies. However, there have been fewer comparative studies and practitioner-oriented guidelines. The ASE community's next quest should be to assess which approaches are most appropriate in specific contexts and explain why this is the case. Answering these fundamental questions could advance existing research while also helping practitioners select appropriate technologies and practices. Obviously, this will require closer collaboration between the automotive and research communities. 

References

1. A. Pretschner et al., "Software Engineering for Automotive Systems: A Roadmap," *Proc. 2007 Future of Software Eng. (FOSE 07)*, 2007, pp. 55–71.
2. A. Haghighatkah et al., "Automotive Software Engineering: A Systematic Mapping Study," *J. Systems and Software*, vol. 128, no. C, 2017, pp. 25–55.
3. S. Dersten, J. Axelsson, and J. Froberg, "Effect Analysis of the Introduction of AUTOSAR: A Systematic Literature Review," *Proc. 37th EUROMICRO Conf. Software Eng. and Advanced Applications (SEAA 11)*, 2011, pp. 239–246.
4. S. Martínez-Fernández et al., "A Survey on the Benefits and Drawbacks of AUTOSAR," *Proc. 1st Int'l Workshop Automotive Software Architecture (WASA 15)*, 2015, pp. 19–26.
5. A. Kasoju, K. Petersen, and M.V. Mäntylä, "Analyzing an Automotive Testing Process with Evidence-Based Software Engineering," *Information and Software Technology*, vol. 55, no. 7, 2013, pp. 1237–1259.
6. J. Grossmann et al., "TestML—a Test Exchange Language for Model-Based Testing of Embedded Software," *Model-Driven Development of Reliable Automotive Services*, LNCS 4922, Springer, 2006, pp. 98–117.
7. A. Pretschner et al., "One Evaluation of Model-Based Testing and Its Automation," *Proc. 27th Int'l Conf. Software Eng. (ICSE 05)*, 2005, pp. 392–401.
8. H. Altinger, F. Wotawa, and M. Schurius, "Testing Methods Used in the Automotive Industry: Results from a Survey," *Proc. 2014 Workshop Joining Academia and Industry Contributions to Test Automation and Model-Based Testing (JAMAICA 14)*, 2014, pp. 1–6.
9. M. Lochau et al., "Model-Based Pairwise Testing for Feature Interaction Coverage in Software Product Line Engineering," *Software Quality J.*, vol. 20, nos. 3–4, 2012, pp. 567–604.
10. R. Matinnejad et al., "Search-Based Automated Testing of Continuous Controllers: Framework, Tool Support, and Case Studies," *Information and Software Technology*, vol. 57, 2015, pp. 705–722.



ALIREZA HAGHIGHATKAH is a PhD candidate in software engineering at the University of Oulu. Contact him at alireza.haghighatkah@oulu.fi.



MARKKU OIVO is a full professor and the director of the M3S (Empirical Software Engineering in Software, Systems and Services) group at the University of Oulu. Contact him at markku.oivo@oulu.fi.



AHMAD BANIJAMALI is a PhD candidate in software engineering at the University of Oulu. Contact him at ahmad.banijamali@oulu.fi.



PASI KUVAJA is an adjunct professor and cofounder of the University of Oulu's M3S group. Contact him at pasi.kuvaja@oulu.fi.

Workshop Continuous Software Evolution and Delivery (CSED 16), 2016, pp. 34–40.

15. R. Rana et al., “Selecting Software Reliability Growth Models and Improving their Predictive Accuracy Using Historical Projects Data,” *J. Systems and Software*, vol. 98, no. C, 2014, pp. 59–78.
16. R. Rana et al., “Defect Prediction over Software Life Cycle in Automotive Domain State of the Art and Road Map for Future,” *Proc. 9th Int’l Conf. Software Eng. and Applications (ICSOFTEA)*, 2014, pp. 377–382.
17. F. Deissenboeck et al., “Model Clone Detection in Practice,” *Proc. 4th Int’l Workshop Software Clones (IWSC 10)*, 2010, pp. 57–64.
18. S. Thiel et al., “Software Product Lines in Automotive Systems Engineering,” *SAE Int’l J. Passenger Cars—Electronic and Electrical Systems*, vol. 1, no. 1, 2009, pp. 531–543.

11. P. Braun et al., “Guiding Requirements Engineering for Software-Intensive Embedded Systems in the Automotive Industry,” *Computer Science—Research and Development*, vol. 29, no. 1, 2014, pp. 21–43.
12. P. Hohl et al., “Forces That Prevent Agile Adoption in the Automotive Domain,” *Proc. 17th Int’l Conf. Product-Focused Software Process*

Improvement (PROFES 16), 2016, pp. 468–476.

13. H. Giese et al., “Model-Based Integration,” *Proc. 2007 Int’l Dagstuhl Conf. Model-Based Eng. of Embedded Real-Time Systems (MBEERTS 07)*, 2010, pp. 17–54.
14. S. Vöst and S. Wagner, “Trace-Based Test Selection to Support Continuous Integration in the Automotive Industry,” *Proc. IEEE/ACM Int’l*

myCS

Read your subscriptions through the myCS publications portal at

<http://mycs.computer.org>